



CS4001NT Programming

60% Individual Coursework

Autumn 2025/26

Student Name: Shrawan Karki

London Met ID: 25028798

College ID: np05cp4a250078

Assignment Due Date: Friday, May 15, 2026

Assignment Submission Date: Friday, May 15, 2026

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1	Introduction	1
1.1	Aims and Objectives	1
1.2	Technology Used	2
1.2.1	Java	2
1.2.2	BlueJ.....	3
1.2.3	Draw.io.....	4
2	Class Diagram.....	5
2.1	AIModel Class Diagram	5
2.2	PersonalPlan Class Diagram	7
2.3	ProPlan class Diagram	7
2.4	SubscriptionGUI class Diagram	8
2.5	Complete UML class diagram	9
3	Pseudocode	10
3.1	AIModel.....	10
3.2	PersonalPlan.....	11
3.3	ProPlan	13
3.4	SubscriptionGUI.....	15
4	Method Description	19
4.1	Abstract class: AIModel.java	19
4.1.1	AIModel.....	19
4.1.2	showDetails	19
4.1.3	processPrompt.....	20
4.2	Sub Class: PersonalPlan.java.....	20
4.2.1	Personal Plan (Constructor).....	20
4.2.2	processPrompt.....	21
4.2.3	showDetails	21
4.3	Sub Class: ProPlan.java	22
4.3.1	ProPlan (Constructor)	22
4.3.2	processPrompt.....	22
4.3.3	Include Member	23
4.4	GUI Class: SubscriptionGUI.java	24

4.4.1	addPersonalLogic	24
4.4.2	addPrologic.....	24
4.4.3	processPromptLogic	25
4.4.4	checkPlan type	25
4.4.5	Export to txt.....	26
4.4.6	Loadfrom Txt.....	26
4.4.7	DisplayLogic	27
4.4.8	getIndex.....	27
4.4.9	clearLogic	28
4.4.10	showError	28
4.4.11	isAnyEmpty.....	29
4.4.12	styleButton	29
4.4.13	setupPanel.....	30
5	Testing	31
5.1	Compile and run program using CLI	31
5.2	Add personal plan to Array list	33
5.3	Add pro plan to Array list.....	35
5.4	Make a Prompt.....	37
5.5	Add team member	39
5.6	Check Plan type using instanceof	41
5.7	Show message dialog for invalid index value.....	43
5.8	Successful file writing (Export)	45
5.9	Successful loading of file created earlier.....	47
6	Error Detection and Correction.....	48
6.1	Syntax Error	48
6.2	Logical Error.....	49
6.3	Runtime Error.....	50
7	Conclusion	51
8	Bibliography	52
9	Appendix	53
9.1	AIModel.java	53
9.2	PersonalPlan.java	56

9.3	ProPlan.java.....	58
9.4	SubscriptionGUI.java	60

Table of figures

Figure 1 : Java logo.....	2
Figure 2 :BlueJ	3
Figure 3 : Draw.io	4
Figure 4 : BlueJ class diagram	5
Figure 5 : AIModel.....	6
Figure 6 : PersonPlan.....	7
Figure 7: Pro Plan	7
Figure 8: SubscriptionGUI	8
Figure 9 :UML class diagram	9
Figure 10 : Command to compile and run program using CLI.....	31
Figure 11 : GUI window.....	32
Figure 12 : Filled monthly details and monthly limit.....	33
Figure 13 : Personal plan added to successfully	34
Figure 14 : Pro Plan creation.....	35
Figure 15 : Pro Plan created successfully	36
Figure 16 : Processing prompt	37
Figure 17 : Remaining quota deduction.....	38
Figure 18 : Adding team member	39
Figure 19 : Team member added Successfully	40
Figure 20 : Checking Plan type using index of Personal Plan	41
Figure 21 : Successfully checked plan type	42
Figure 22 : Plan Check.....	42
Figure 23 : Index out of Bounds Check	43
Figure 24 : Successfully displayed error message	44
Figure 25 : Clicking export to txt.....	45
Figure 26 : Message popup exported to txt	46
Figure 27 : Exported to subscriptions.txt	46
Figure 28 : text file.....	46
Figure 29 : Successfully loaded from exported file	47
Figure 30 : Screenshot of Syntax error.....	48
Figure 31 : Screenshot of solution to syntax error	48
Figure 32 : Logical error encounter	49
Figure 33 : Runtime error	50
Figure 34 :Exception handling.....	50

Table of tables

Table 1 : AIModel (Constructor) Method Description.....	19
Table 2 : showDetails Method Description	19
Table 3 : processPrompt abstract method AIModel Method Description.....	20
Table 4 : Personal Plan (Constructor)	20
Table 5 : processPrompt abstract method PersonalPlan Method Description.....	21
Table 6 : showDetails personal plan Method Description.....	21
Table 7 : ProPlan (Constructor) Method Description	22
Table 8 : processPrompt Pro plan Method Description	22
Table 9 : Add member Method Description	23
Table 10 : addPersonalLogic Method Description	24
Table 11 : addPrologic Method Description.....	24
Table 12 : processPrompt Method Description	25
Table 13 : CheckPlan Type Method Description	25
Table 14 : Export to text file method Description.....	26
Table 15 : Load From txt method Description	26
Table 16 : Display Logic Method Description	27
Table 17 : getIndex Method Description.....	27
Table 18 : clearLogic Method Description	28
Table 19 : ShowError Method Description.....	28
Table 20 : isAnyEmpty Method Dessionction.....	29
Table 21 : styleButton Method Description.....	29
Table 22 : setupPanel method description	30
Table 23 :Compile and run program using CLI.....	31
Table 24 : Add personal plan to Arraylist.....	33
Table 25 : Add pro plan to Array list	35
Table 26 : Make Prompt.....	37
Table 27: Add team member.....	39
Table 28 : Check plan type using instanceof.....	41
Table 29 : Invalid index error	43
Table 30 : Successful file writing (Export)	45
Table 31: loading earlier file	47

1 Introduction

The UML diagrams and source codes elaborate further on the AI Model Subscription Management System that has been created in the Java programming language. The OOP model comprises many aspects of the fundamental principles of OOP. Among these aspects of the OOP model is the use of the class named AIModel that has properties such as the cost of the token, number of parameters, context windows, and others whose methods include calculating the total number of tokens in the system. The OOP program has been expanded to two other classes including PersonalPlan and ProPlan. Under the PersonalPlan class, the program focuses only on the creation of the plan by the individual creator, assessing prompts on a monthly basis. Conversely, under the ProPlan class, the program assesses plans for a collection of individuals based on their number of subscribers. Furthermore, the program has the GUI capability using Swing in Java.

1.1 Aims and Objectives

- For Developing a Custom Subscription Management Application Using AI
- For Implementing an Adaptive Resource Allocation and Utilization Mechanism
- For Building an Advanced User Interface for End Users
- For Ensuring Data Persistence through Serialization
- For Conducting Real-Time Monitoring and Debugging of Systems

1.2 Technology Used

1.2.1 Java

Among the programming languages which can be termed complex include Java; however, at the same time, it can be said to be versatile. It is an object-oriented programming language created by James Gosling and Sun Microsystems in the year 1995. This programming language had been created in such a manner that there would be no dependency on anything else. That is the reason why Java comes with WORA capabilities. Because of this capability, it can run on any machine which has JVM regardless of the operating system (igmguru, 2026).



Figure 1 : Java logo

1.2.2 BlueJ

BlueJ is an open-source development tool which has been created specially for learning Java programming language. It is used exclusively for educational purposes, being aimed at beginners who want to master OOP in a totally different way. Unlike many other systems, BlueJ uses visual representations rather than giving the user some complicated fragment of code which is difficult to comprehend for someone new to programming. Rather than showing him/her only the code he or she needs to run the application, BlueJ provides a view on all elements of the whole system architecture using visual analogies of the UML with classes represented as blocks communicating with each other. BlueJ is different from other tools in that it features "Object Bench" which lets you create objects, examine them and call their methods without writing any code associated with the main method (mivocloude, 2023).



Figure 2 :BlueJ

1.2.3 Draw.io

I really like using Diagrams.net, which used to be called Draw.io. It is a free tool that you can use on the web to make all sorts of diagrams. You can make lots of diagrams with Draw.io like flowcharts and UML diagrams and network diagrams and organizational charts and mind maps. This is because Draw.io has a drag-and-drop feature with lots of shapes to choose from. One of the things about Draw.io is that you do not have to make an account or pay any money to use it. Draw.io also lets you save your diagrams to lots of cloud storage places, such as Google Drive and OneDrive and Dropbox and GitHub. So you can store your diagrams. Export them when you need to. You can make all sorts of diagrams, with Draw.io, including flowcharts and mind maps and network diagrams and organizational charts and UML diagrams.



Figure 3 : Draw.io

2 Class Diagram

IT is also considered as the structure of a structure of system:

- Class
- Methods
- Attributes
- Relationship

The Class Diagram required for this java milestone are listed below respectively:

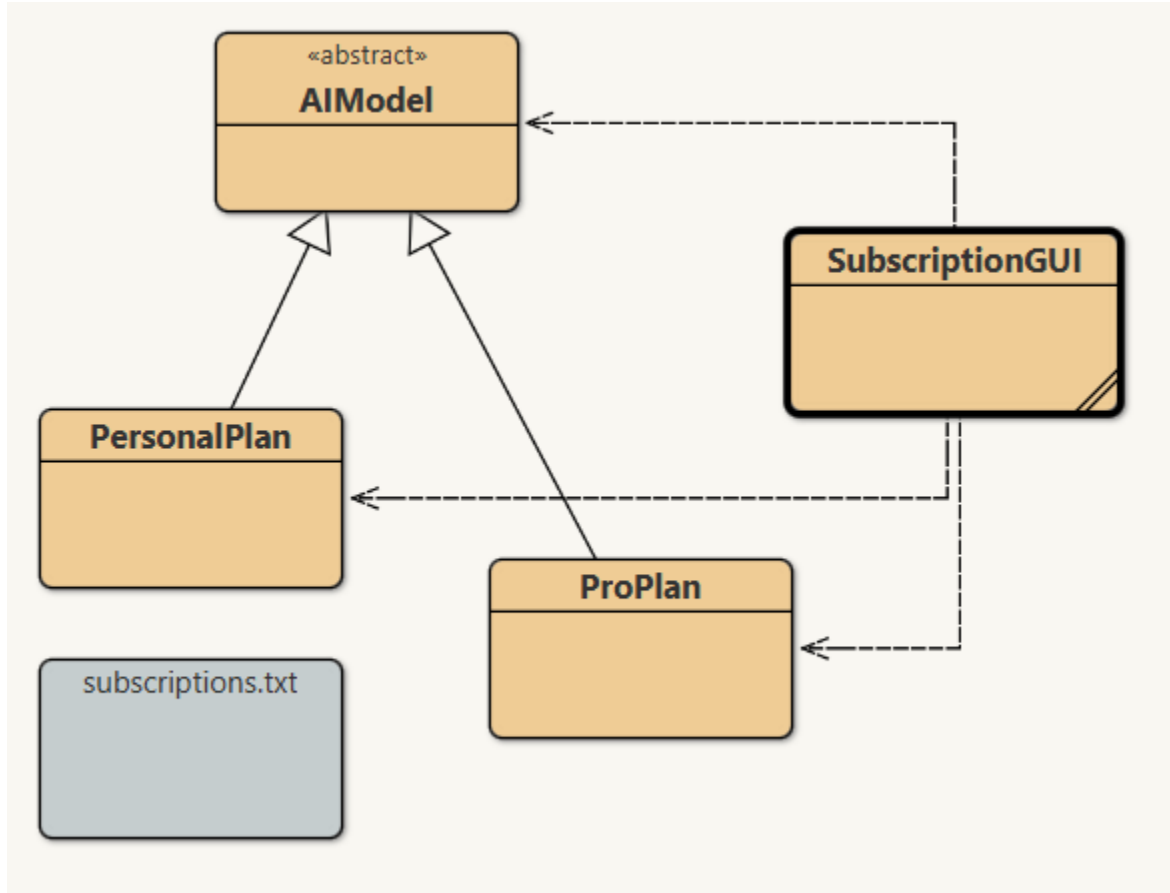


Figure 4 : BlueJ class diagram

2.1 AIModel Class Diagram

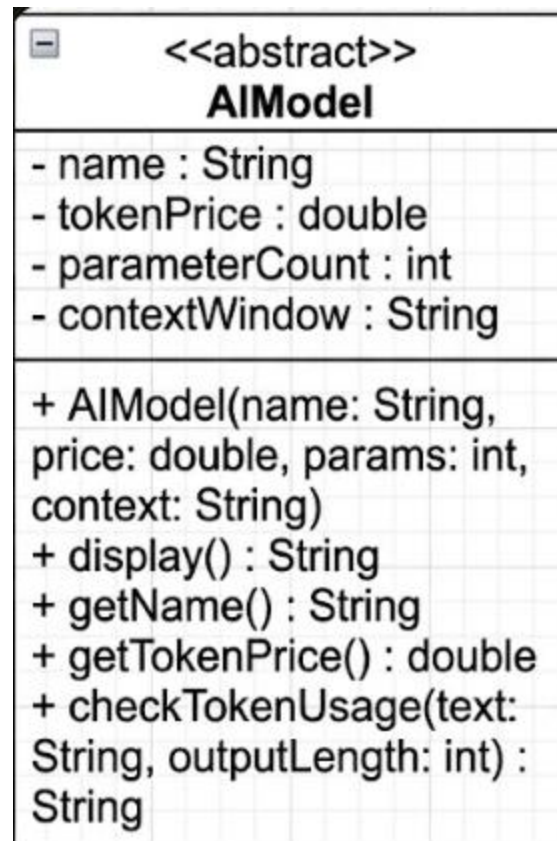


Figure 5 : AIModel

2.2 PersonalPlan Class Diagram

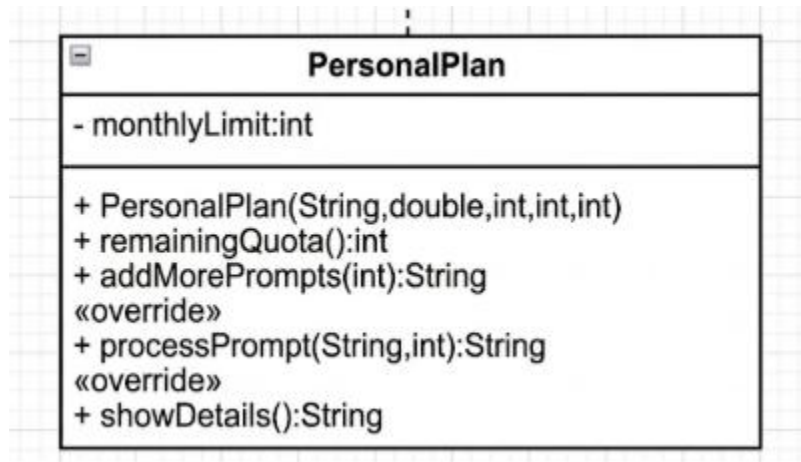


Figure 6 : PersonPlan

2.3 ProPlan class Diagram

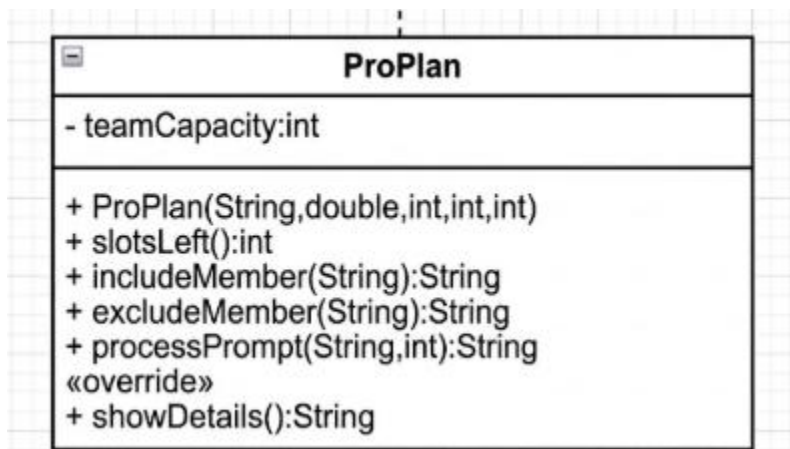


Figure 7: Pro Plan

2.4 SubscriptionGUI class Diagram

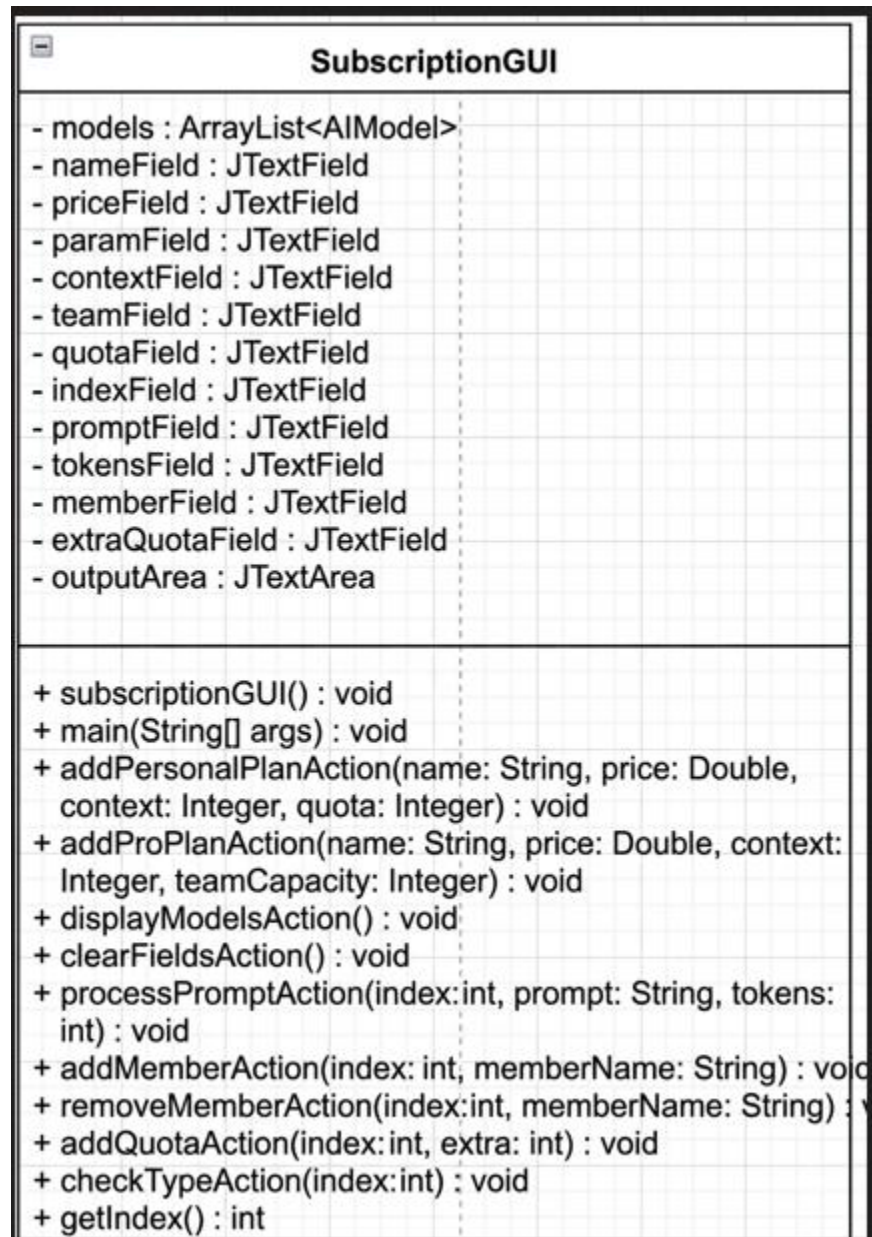


Figure 8: SubscriptionGUI

2.5 Complete UML class diagram

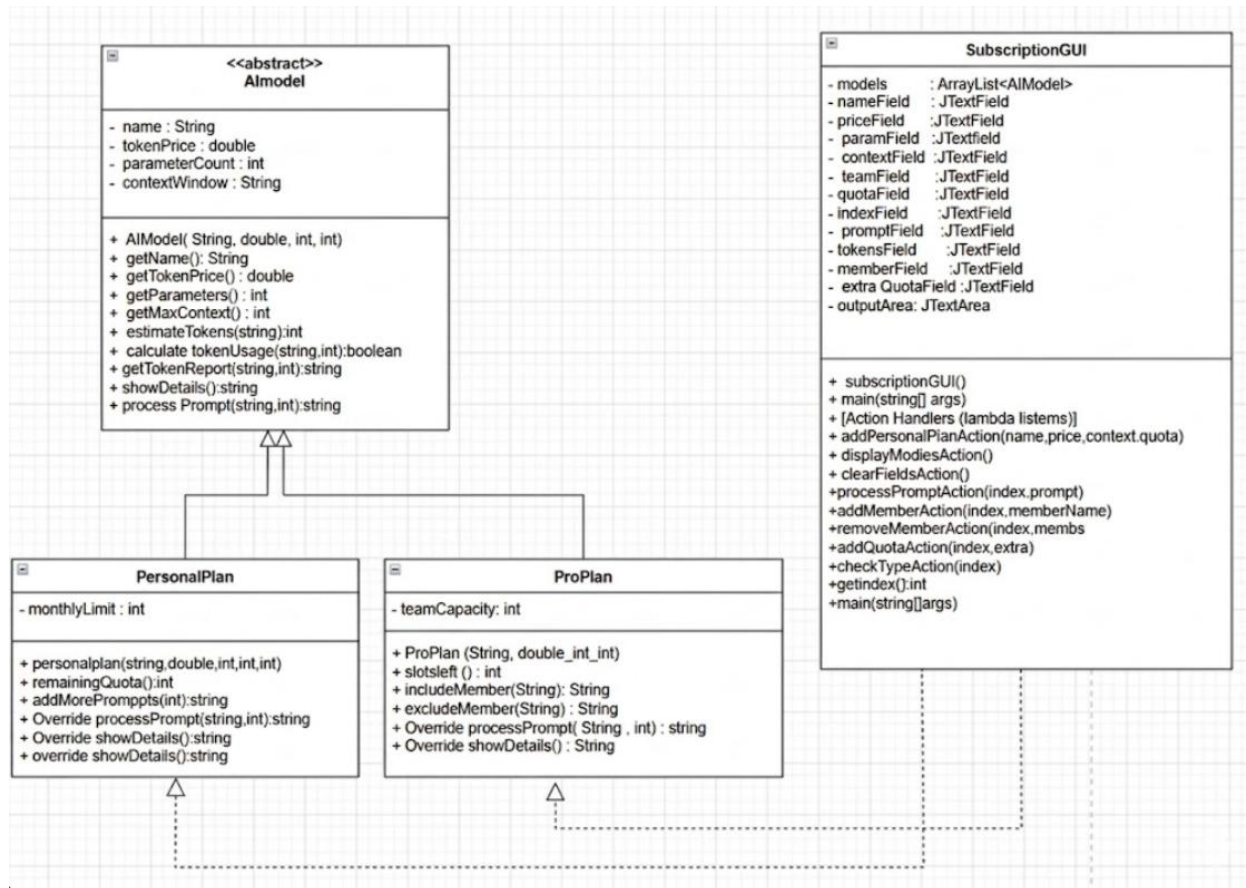


Figure 9 :UML class diagram

3 Pseudocode

Pseudocode can thus be defined as an easy, yet high-level explanation of the steps taken during the development of an algorithm or program by computers. Although there are various elements in pseudocode like conditionals and loops which are prevalent in programming languages, it relies more on human language rather than that of a machine. It makes it easier for programmers not to worry about comprehending complex languages or considering how machines work since their attention will be directed towards the problem-solving process. Pseudocode is frequently utilized by programmers when developing computer applications, getting ideas or concepts across before writing down codes using languages like Python or Java (codecademy, 2024).

3.1 AIModel

AIModel

name: String

tokenPrice: Float

parameters: Integer

maxContext: Integer

CONSTRUCTOR(name, tokenPrice, parameters, maxContext)

 this.name = name

 this.tokenPrice = tokenPrice

 this.parameters = parameters

 this.maxContext = maxContext

END CONSTRUCTOR

FUNCTION estimateTokens(text)

 RETURN length of text / 4

END FUNCTION

FUNCTION calculate Token Usage

```

    total Token = estimate Token + 50 + output Token
    RETURN totalTokens <= maxContext
END FUNCTION

```

```

FUNCTION getTokenReport(text, outputTokens)
    input Token = estimate Token
    total Token = input Token + 50 + output Token

    IF totalTokens <= maxContext THEN
        RETURN report + "\nStatus: WITHIN LIMIT"
    ELSE
        RETURN report + "\nStatus: EXCEEDS LIMIT"
    END IF
END FUNCTION

```

```

ABSTRACT FUNCTION processPrompt(text, tokens)

```

```

FUNCTION showDetails()
    RETURN " Model:" + name + "Price:" + token Price + "Parameters:" + parameter +
    "Max Context: " + max Context
END FUNCTION
END CLASS

```

3.2 PersonalPlan

PersonalPlan

```

CONSTRUCTOR (name, token Price, parameters, max Context, monthly Limit)
    SUPER (name, token Price, parameters, max Context)
    Monthly Limit = monthly Limit

```

End constructor

Function remaining Quota ()

Return monthly limit

End functions

Function adds More Prompt

If done extra prompt < = 0 Then

Return " Invalid input."

End if

Monthly Limit = monthly Limit + extra

RETURN "Quota updated: " + monthly Limit

END FUNCTION

FUNCTION process Prompt (text, tokens)

IF tokens > metacontext () THEN

RETURN "Exceeds context size."

END IF

IF monthly Limit <= 0 THEN

RETURN "No prompts left."

END IF

Monthly Limit = - 1

RETURN " Processed. Remaining Quota: " + monthly Limit

End function

FUNCTION show Details ()

RETURN Super. show Details () + "Prompts Remaining: " + monthly Limit

End functions

End class

3.3 ProPlan

ProPlan

tea

FUNCTION New (name as String, token Price as Integer, parameters as dictionary,
max context as Integer, team capacity as Integer)

CALL SUPER. New (name, token Price, parameters, max Context)

this. Team Capacity = team Capacity

end function

Function slots Available ()

RETURN team Capacity

END FUNCTION

FUNCTION insert Member (member AS String)

IF member IS NULL THEN

RETURN "Member name is invalid."

END IF

IF team Capacity < 0 THEN

RETURN "Team is full."

end

Team Capacity = team Capacity - 1

RETURN member " has been add."

End function

FUNCTION exclude Member (member AS String)

IF member IS NULL THEN

RETURN "Member name is invalid."

END IF

Team Capacity = team Capacity + 1

RETURN member & " has been excluded from the team."

END FUNCTION

```

FUNCTION processPrompt(prompt AS String, tokens AS Integer)
  IF tokens > getMaxContext() THEN
    RETURN "Token number is greater than the maximum context size allowed."
  END IF
  Monthly limit exceeded
END FUNCTION
FUNCTION showDetails()
  RETURN SUPER.showDetails()
END FUNCTION
END CLASS

```

3.4 SubscriptionGUI

```

CLASS SubscriptionGUI EXTENDS JFrame
  models: List of AIModel
  TXT_FILE: String = "subscriptions.txt"
  UI_Elements: TextFields, Buttons, JTextArea

  CONSTRUCTOR()
    Set window title, size, and layout

    Create GlobalPanel with fields for name, price, parameters, context
    Create HubsPanel with fields for quota and team
    Create OperationsPanel with fields for index, prompt, tokens, member
    Create OutputArea

    Add panels to window

    ON AddPersonalButton CLICK:

```

```
CALL addPersonalLogic()
```

```
ON AddProButton CLICK:
```

```
    CALL addProLogic()
```

```
ON PromptButton CLICK:
```

```
    CALL processPromptLogic()
```

```
ON AddMemberButton CLICK:
```

```
    CALL addMemberLogic()
```

```
ON CheckTypeButton CLICK:
```

```
    TRY
```

```
        index = PARSE Integer from indexField
```

```
        CALL checkPlanType(index)
```

```
    CATCH Exception
```

```
        CALL showError("Invalid Index")
```

```
    END TRY
```

```
ON ExportButton CLICK:
```

```
    Write all models details to TXT_FILE
```

```
ON LoadButton CLICK:
```

```
    Read from TXT_FILE and print to OutputArea
```

```
ON DisplayButton CLICK:
```

```
    CALL displayLogic()
```

ON ClearButton CLICK:

CALL clearLogic()

Show Window

END CONSTRUCTOR

FUNCTION checkPlanType(index)

IF index < 0 OR index >= models.size THEN

CALL showError("Index Out of Bounds")

RETURN

END IF

selectedPlan = models.get(index)

IF selectedPlan IS TYPE OF PersonalPlan THEN

Print "Personal Plan" to OutputArea and Dialog

ELSE IF selectedPlan IS TYPE OF ProPlan THEN

Print "Pro Plan" to OutputArea and Dialog

ELSE

Print "General AI Model" to OutputArea

END IF

END FUNCTION

FUNCTION setupPanel(panel, title) END FUNCTION

FUNCTION styleButton(button) END FUNCTION

FUNCTION createWhiteLabel(text) END FUNCTION

FUNCTION addLabelAndField(panel, text, field) END FUNCTION

```
FUNCTION addPersonalLogic() END FUNCTION  
FUNCTION addProLogic() END FUNCTION  
FUNCTION processPromptLogic() END FUNCTION  
FUNCTION addMemberLogic() END FUNCTION  
FUNCTION displayLogic() END FUNCTION  
FUNCTION clearLogic() END FUNCTION  
FUNCTION showError(msg) END FUNCTION
```

```
MAIN FUNCTION
```

```
    CREATE SubscriptionGUI
```

```
END MAIN FUNCTION
```

```
END CLASS
```

4 Method Description

A short explanation of what the method does is termed as method description (Werner, 2019).

- Brief description of the method purpose
- List the parameters
- Describe the return type
- Include any exception that may be thrown

4.1 Abstract class: AIModel.java

4.1.1 AIModel

Method Name	AIModel (Constructor)
Method Description	Initialize basic layout structure of AI model
Parameters	String modelName, double tokenPrice, int parameters, int maxContent
Return Type	None
Logic	Assign values from parameters to class instance variable use Keyword 'this'.

Table 1 : AIModel (Constructor) Method Description

4.1.2 showDetails

Method Name	showDetails
Method Description	Provide string to model's data
Parameters	None
Return Type	String
Logic	Concatenate name, price, parameter and context value to a formatted readable string.

Table 2 : showDetails Method Description

4.1.3 processPrompt

Method Name	processPrompt
Method Description	Abstract method for prompt simulation
Parameters	String text, int tokens
Return Type	String
Logic	Specifies and ensures all specific plans implement their own version of their logic

Table 3 : processPrompt abstract method AIModel Method Description

4.2 Sub Class: PersonalPlan.java

4.2.1 Personal Plan (Constructor)

Method Name	PersonalPlan (Constructor)
Method Description	Create a personal subscription plan
Parameters	String modelName, double tokenPrice, int parameters, int maxContent, int monthlyQuota
Return Type	None
Logic	Call super constructor and initialize to monthly quota field

Table 4 : Personal Plan (Constructor)

4.2.2 processPrompt

Method Signature	processPrompt(String text, int tokens)
Method Description	Implement prompt process with a decrement monthly quota decrement system
Parameters	String text – the prompt Int tokens – requested tokens
Return type	String- Returns processing details or an “Out of quota” error
Logic	• Checks if token exceed context. • Checks if monthly limit is greater than 0. • If both pass decrement monthly limit by 1, then return a success with the remaining quota

*Table 5 : processPrompt abstract method PersonalPlan Method Description***4.2.3 showDetails**

Method Name	showDetails
Method Description	Extends details report personal plans
Parameters	None
Return Type	String
Logic	Call super.showDetails() and appends the specific monthly prompt limit

Table 6 : showDetails personal plan Method Description

4.3 Sub Class: ProPlan.java

4.3.1 ProPlan (Constructor)

Method Name	ProPlan (Constructor)
Method Description	Sets team-based plan
Parameters	String modelName, double price, int parameters, int maxContent, int membersCount
Return Type	None
Logic	Instance the member ArrayList, initialize team based professional plan

Table 7 : ProPlan (Constructor) Method Description

4.3.2 processPrompt

Method Signature	processPrompt(String text, int tokens)
Method Description	Implements high priority prompt processing without quota restrictions.
Parameters	String text – The prompt Int tokens – Requested tokens
Return type	String – Return processing confirmation
Logic	Unlike Personal plan it does not limit it validates the context size and return high priority success message.

Table 8 : processPrompt Pro plan Method Description

4.3.3 Include Member

Method Signature	includeMember(String name)
Method Description	Adds a new user to professional team list
Parameters	String name – Name of member to add
Return type	String – Confirmation message or error
Logic	Checks if the current size of team members list is less than team capacity if space exists the name is added to Array list.

Table 9 : Add member Method Description

4.4 GUI Class: SubscriptionGUI.java

4.4.1 addPersonalLogic

Method name	addPersonalLogic
Method Description	PersonalPlan object based on user input.
Parameters	None
Return type	Void
Logic	Ensure that inputs are not empty and are numeric. Initialize a PersonalPlan object and adds it to the model ArrayList.

Table 10 : addPersonalLogic Method Description

4.4.2 addPrologic

Method name	addProLogic
Method Description	Handles the creation and storage of a new ProPlan object based on user input.
Parameters	None
Return type	Void
Logic	it creates a ProPlan instance and stores it in the model's collection.

Table 11 : addPrologic Method Description

4.4.3 processPromptLogic

Method name	processPromptLogic
Method Description	Simulates the execution of an AI prompt for a selected model from the registry.
Parameters	None
Return type	Void
Logic	It calls the processPrompt method on the object at the specified index in the ArrayList and displays the resulting string in the output area.

Table 12 : processPrompt Method Description

4.4.4 checkPlan type

Method name	checkPlanType
Method Description	Identifies the specific subclass of a stored AIModel object.
Parameters	int index
Return type	Void
Logic	object at the given index. uses the 'instanceof' operator to check if the object is a PersonalPlan or ProPlan and pop up message dialog to inform the user.

Table 13 : CheckPlan Type Method Description

4.4.5 Export to txt

Method name	exportToTxt
Method Description	Saves the current state of the model to a permanent text file.
Parameters	None
Return type	Void
Logic	Iterates through the models ArrayList. It uses a PrintWriter and FileWriter to write the output of each model's showDetails() method into 'subscriptions.txt'.

*Table 14 : Export to text file method Description***4.4.6 Loadfrom Txt**

Method name	loadFromTxt
Method Description	Retrieves and displays previously saved data from the text file.
Parameters	None
Return type	Void
Logic	Initializes a BufferedReader to read 'subscriptions.txt'. It reads the file line-by-line and appends the content to the GUI's JTextArea for user prereview.

Table 15 : Load From txt method Description

4.4.7 DisplayLogic

Method name	displayLogic
Method Description	display to show all current subscription plans.
Parameters	None
Return type	Void
Logic	Clears the output text area. It then loops through the entire models ArrayList, calling showDetails() on each object and showcasing the result to the display area.

*Table 16 : Display Logic Method Description***4.4.8 getIndex**

Method name	getIndex
Method Description	A helper method to safely retrieve and validate the index input.
Parameters	None
Return type	int
Logic	Reads text from the indexField. It uses a try-catch block to handle NumberFormatException. If valid, it returns the integer; otherwise, it shows an error and returns -1.

Table 17 : getIndex Method Description

4.4.9 clearLogic

Method name	clearLogic
Method Description	Clears all input and output components in the user interface.
Parameters	None
Return type	Void
Logic	Iterates through all JTextField components and sets their text to an empty string. It also clears the JTextArea log.

*Table 18 : clearLogic Method Description***4.4.10 showError**

Method name	showError
Method Description	display of error messages to the user.
Parameters	String msg
Return type	Void
Logic	Invokes JOptionPane.showMessageDialog with the error message icon to present a popup containing the error details passed.

Table 19 : ShowError Method Description

4.4.11 isEmpty

Method name	isEmpty
Method Description	Validate field completion before processing data.
Parameters	TextField
Return type	boolean
Logic	Loop around provided array of text fields and checks if any call returns. Returns true if a blank field is found.

*Table 20 : isEmpty Method Description***4.4.12 styleButton**

Method name	styleButton
Method Description	Applies the dark-mode theme to GUI buttons.
Parameters	JButton button
Return type	Void
Logic	Sets the background color to dark grey, the text to white.

Table 21 : styleButton Method Description

4.4.13 setupPanel

Method name	setupPanel
Method Description	Orders panel gui arrangement
Parameters	JPanel panel, String title
Return type	Void
Logic	Sets the panel's background color and adds a TitledBorder with white text to logically group related input fields.

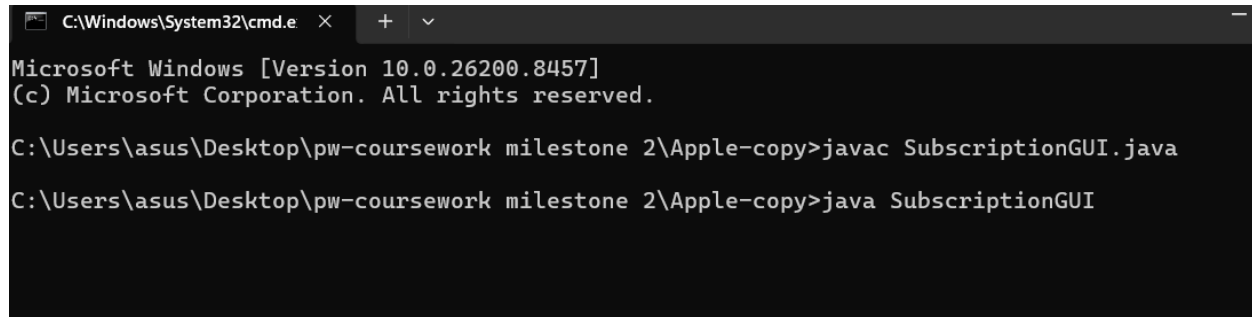
Table 22 : setupPanel method description

5 Testing

5.1 Compile and run program using CLI

Objective	Compile and run program using Command line Interface (CLI)
Action	• Open Terminal / CLI • Navigate to project folder • Type javac SubscriptionGUI.java • Then java SubscriptionGUI
Expected Output	The program should compile successfully, and the GUI window should appear
Actual Output	The GUI window appeared
Result	The test was successful

Table 23 :Compile and run program using CLI



```

C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\asus\Desktop\pw-coursework milestone 2\Apple-copy>javac SubscriptionGUI.java

C:\Users\asus\Desktop\pw-coursework milestone 2\Apple-copy>java SubscriptionGUI

```

Figure 10 : Command to compile and run program using CLI

AI Model Subscription System - Shrawan karki

Model Details

Model Name: Price (NPR):

Parameters (B): Max Context:

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

Figure 11 : GUI window

5.2 Add personal plan to Array list

Objective	Add personal plan to Arraylist
Action	• Fill Model details • Fill Monthly limit • Click Add Personal Plan button
Expected Output	The plan is stored in the ArrayList and a confirmation message appears in the log.
Actual Output	Personal plan was added to Arraylist
Result	The test was successful

Table 24 : Add personal plan to Arraylist

The screenshot shows a web application titled "AI Model Subscription System - Shrawan karki". The "Model Details" section contains the following fields and values:

- Model Name:** Personal Plan Test
- Price (NPR):** 150
- Parameters (B):** 175
- Max Context:** 128000

Below these fields are two columns for plan details:

- Personal Plan:**
 - Monthly Prompt Limit:** 500
 - Add Personal Plan** (button)
- Pro Plan:**
 - Team Capacity:** (empty field)
 - Add Pro Plan** (button)

At the bottom, the "Operations and File Management" section includes:

- Target Index:** (empty field)
- Prompt:** (empty field)
- Tokens:** (empty field)
- Process Prompt** (button)

Figure 12 : Filled monthly details and monthly limit

AI Model Subscription System - Shrawan karki

Model Details

Model Name:	Personal Plan Test	Price (NPR):	150
Parameters (B):	175	Max Context:	128000

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

SYSTEM: Personal Plan added successfully at Index 0

Figure 13 : Personal plan added to successfully

5.3 Add pro plan to Array list

Objective	Add prol plan to Arraylist
Action	• Fill Model details • Fill Team Capacity • Click Add Pro Plan button
Expected Output	The plan is stored in the ArrayList and a confirmation message appears in the log.
Actual Output	Pro plan was added to Arraylist
Result	The test was successful

Table 25 : Add pro plan to Array list

The screenshot shows a web application window titled "AI Model Subscription System - Shrawan karki". The interface is divided into several sections:

- Model Details:**
 - Model Name:
 - Price (NPR):
 - Parameters (B):
 - Max Context:
- Personal Plan:**
 - Monthly Prompt Limit:
 -
- Pro Plan:**
 - Team Capacity:
 -
- Operations and File Management:** (A section at the bottom, currently empty)

Figure 14 : Pro Plan creation

AI Model Subscription System - Shrawan karki

Model Details

Model Name:	Pro Plan Test	Price (NPR):	250
Parameters (B):	200	Max Context:	200000

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

SYSTEM: Pro Plan added successfully at Index 1

Figure 15 : Pro Plan created successfully

5.4 Make a Prompt

Objective	Make a prompt
Action	• Enter valid index example 0 that is personal plan • Enter prompt text and token count • Click Process Prompt button
Expected Output	The system calculates remaining quota and displays the Success or failure message.
Actual Output	The system successful calculated remaining token and prompt count
Result	The test was successful

Table 26 : Make Prompt

The screenshot displays a web application interface titled "Operations and File Management". It features several input fields and buttons for managing prompts. The "Target Index" field is set to "0", the "Prompt" field contains the text "Write an essay on sun", and the "Tokens" field is set to "50". A "Process Prompt" button is located to the right of the "Tokens" field. Below these, there is a "Member Name" input field with an "Add Member" button. Further down, a "Check Plan at Index Above:" label is followed by a "Check Plan Type" button. At the bottom, there are four buttons: "Export to TXT", "Load From TXT", "Display All", and "Clear All Fields".

Figure 16 : Processing prompt

AI Model Subscription System - Shrawan karki

Model Details

Model Name: Price (NPR):

Parameters (B): Max Context:

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

LOG: Processing Index 0...
Processing Prompt...
Text: Write an essay on sun
Tokens Used: 50
Remaining Quota: 499

Figure 17 : Remaining quota deduction

5.5 Add team member

Objective	Add Team member
Action	• Enter Pro plan index • Enter name in the member field • Click Add Member
Expected Output	The member plan is added to the pro plan list of members
Actual Output	Member added successful
Result	The test was successful

Table 27: Add team member

The screenshot shows a software interface titled "Operations and File Management". It contains several input fields and buttons. The "Target Index" field is set to "1". The "Prompt" field is empty. The "Tokens" field is empty. The "Member Name" field is set to "Raju Mistri". The "Add Member" button is highlighted with a blue border. Below this, there is a "Check Plan at Index Above:" label and a "Check Plan Type" button. At the bottom, there are four buttons: "Export to TXT", "Load From TXT", "Display All", and "Clear All Fields".

Figure 18 : Adding team member

AI Model Subscription System - Shrawan karki

Model Details

Model Name: Price (NPR):

Parameters (B): Max Context:

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

Raju Mistri successfully joined the team.

Figure 19 : Team member added Successfully

5.6 Check Plan type using instanceof

Objective	Check plan type
Action	• Enter a valid index • Click check Plan type button
Expected Output	A message box appears identifying whether it is Pro Plan or Personal Plan
Actual Output	Message box appeared
Result	The test was successful

Table 28 : Check plan type using instanceof

The screenshot shows a software interface titled "Operations and File Management". It contains several input fields and buttons. The "Target Index" field is set to "0". The "Prompt" field is empty. The "Tokens" field is empty. The "Process Prompt" button is visible. The "Member Name" field is empty, and the "Add Member" button is next to it. The "Check Plan at Index Above:" label is followed by a button labeled "Check Plan Type", which is highlighted with a red border. At the bottom, there are four buttons: "Export to TXT", "Load From TXT", "Display All", and "Clear All Fields".

Figure 20 : Checking Plan type using index of Personal Plan

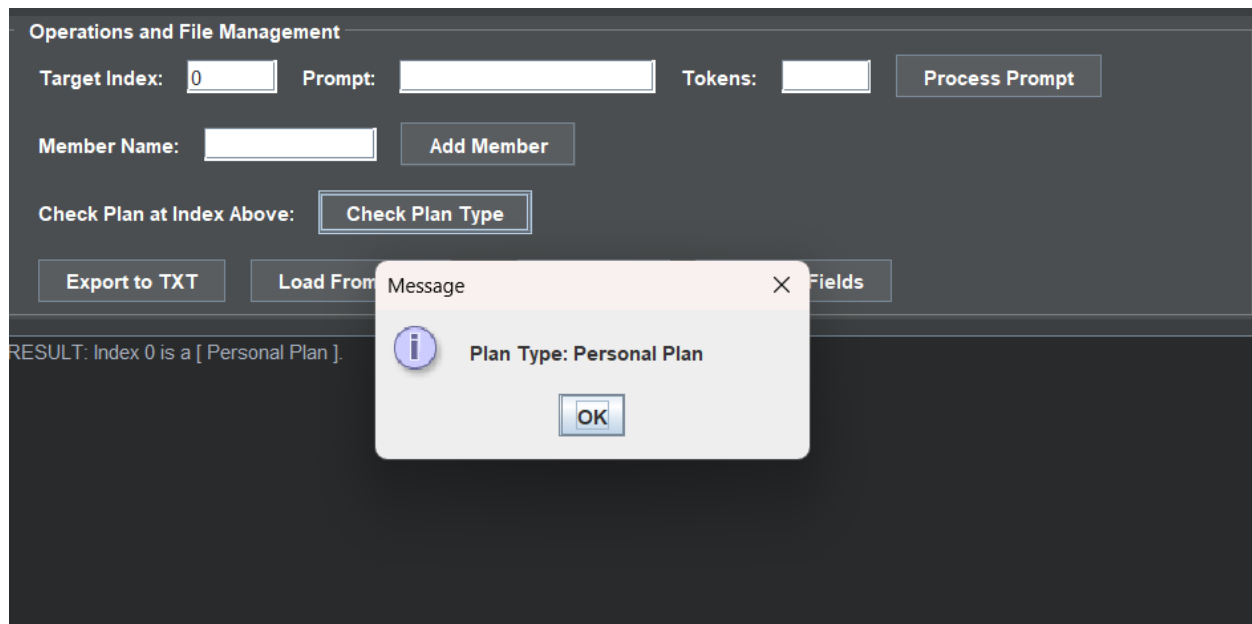


Figure 21 : Successfully checked plan type

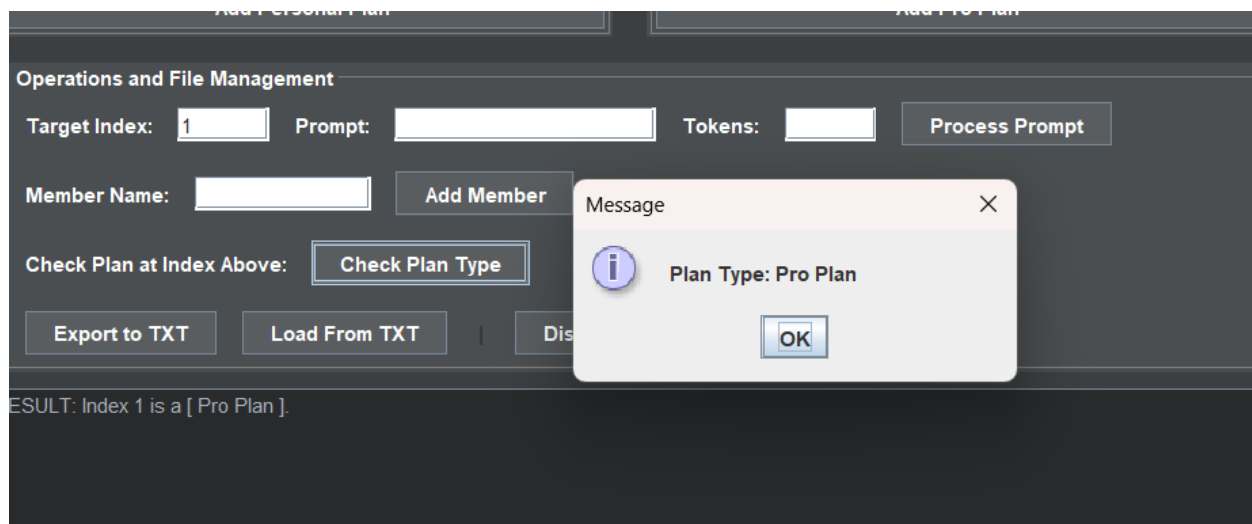


Figure 22 : Plan Check

5.7 Show message dialog for invalid index value

Objective	Show dialog box for invalid index value
Action	- Enter index that does not exist example 7 - Click Check Plan type or Process Prompt
Expected Output	An error dialog appears displaying Invalid Index or Index Out of Bounds
Actual Output	The dialog box appeared
Result	The test was successful

Table 29 : Invalid index error

The screenshot shows the 'AI Model Subscription System - Shrawan karki' window. It features several sections: 'Model Details' with fields for Model Name, Parameters (B), Price (NPR), and Max Context; 'Personal Plan' and 'Pro Plan' sections with fields for Monthly Prompt Limit and Team Capacity, each with an 'Add' button; and 'Operations and File Management' at the bottom. In the 'Operations and File Management' section, the 'Target Index' field is set to '7', and the 'Check Plan at Index Above:' button is highlighted with a red box. Other buttons like 'Process Prompt', 'Add Member', 'Check Plan Type', 'Export to TXT', 'Load From TXT', 'Display All', and 'Clear All Fields' are also visible.

Figure 23 : Index out of Bounds Check

AI Model Subscription System - Shrawan karki

Model Details

Model Name: Price (NPR):

Parameters (B): Max Context:

Personal Plan

Monthly Prompt Limit:

Pro Plan

Team Capacity:

Operations and File Management

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

Alert

 Index Out of Bounds! No plan exists at index 7

Figure 24 : Successfully displayed error message

5.8 Successful file writing (Export)

Objective	File Export
Action	• Ensure at least one plan is added • Click export button
Expected Output	A subscriptions.txt file is created with readable plan data
Actual Output	The text file was created
Result	The test was successful

Table 30 : Successful file writing (Export)

The screenshot displays the 'AI Model Subscription System - Shrawan karki' window. It features several sections: 'Model Details' with input fields for Model Name, Price (NPR), Parameters (B), and Max Context; 'Personal Plan' and 'Pro Plan' sections with input fields for Monthly Prompt Limit and Team Capacity, and 'Add' buttons; 'Operations and File Management' with fields for Target Index, Prompt, and Tokens, and buttons for Process Prompt, Add Member, Check Plan Type, Export to TXT, Load From TXT, Display All, and Clear All Fields; and a 'CURRENT REGISTERED MODELS' section listing two models with their respective details.

Model Details

Model Name: Price (NPR):

Parameters (B): Max Context:

Personal Plan

Monthly Prompt Limit:

Add Personal Plan

Pro Plan

Team Capacity:

Add Pro Plan

Operations and File Management

Target Index: Prompt: Tokens: Process Prompt

Member Name: Add Member

Check Plan at Index Above: Check Plan Type

Export to TXT Load From TXT Display All Clear All Fields

--- CURRENT REGISTERED MODELS ---

Index [0]: AI Model: Personal Plan Test
Token Price (per 100K): NPR 150.0
Parameters (Billion): 175
Max Context: 128000 tokens
Monthly Prompts Remaining: 499

Index [1]: AI Model: Pro Plan Test
Token Price (per 100K): NPR 250.0
Parameters (Billion): 200
Max Context: 200000 tokens
Team Slots Available: 9

Figure 25 : Clicking export to txt

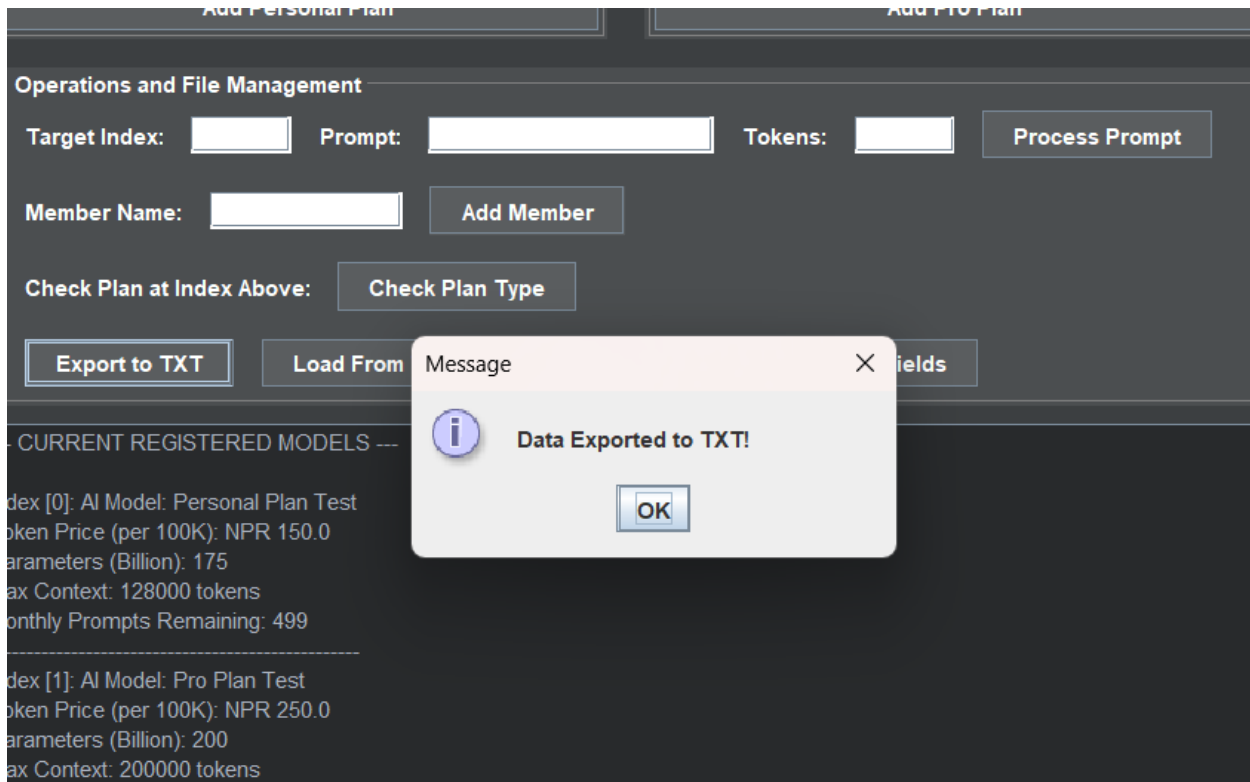


Figure 26 : Message popup exported to txt

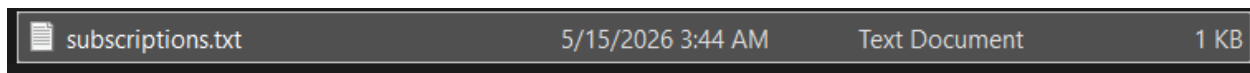


Figure 27 : Exported to subscriptions.txt

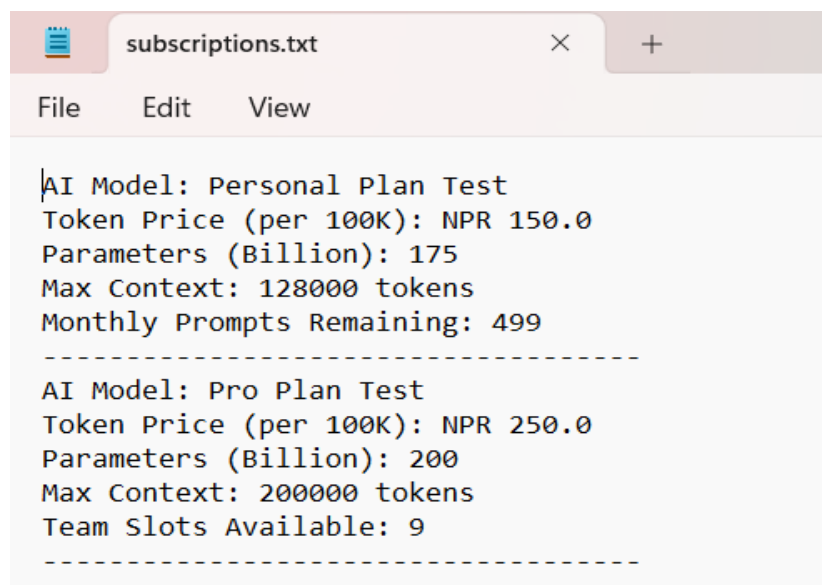


Figure 28 : text file

5.9 Successful loading of file created earlier

Objective	Successful loading of file created earlier
Action	Click load button
Expected Output	The contents of the text file are displayed accurately in the GUI text area.
Actual Output	Contents were displayed in display area
Result	The test was successful

Table 31: loading earlier file

Target Index: Prompt: Tokens:

Member Name:

Check Plan at Index Above:

|

--- RETRIEVED FROM FILE ---

AI Model: Personal Plan Test
 Token Price (per 100K): NPR 150.0
 Parameters (Billion): 175
 Max Context: 128000 tokens
 Monthly Prompts Remaining: 499

AI Model: Pro Plan Test
 Token Price (per 100K): NPR 250.0
 Parameters (Billion): 200
 Max Context: 200000 tokens
 Team Slots Available: 9

Figure 29 : Successfully loaded from exported file

6 Error Detection and Correction

6.1 Syntax Error

Screenshot of Error

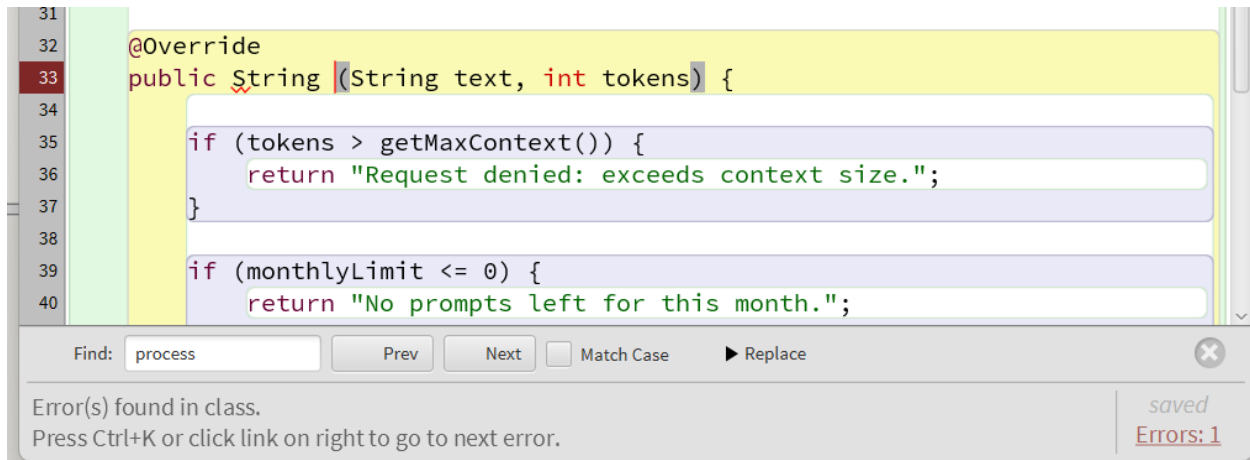


Figure 30 : Screenshot of Syntax error

Cause:

Missing the processprompt due to which it was unable to calculate the quota.

Solution:

Adding processprompt help to fix the problem.

Screenshot of Solution:

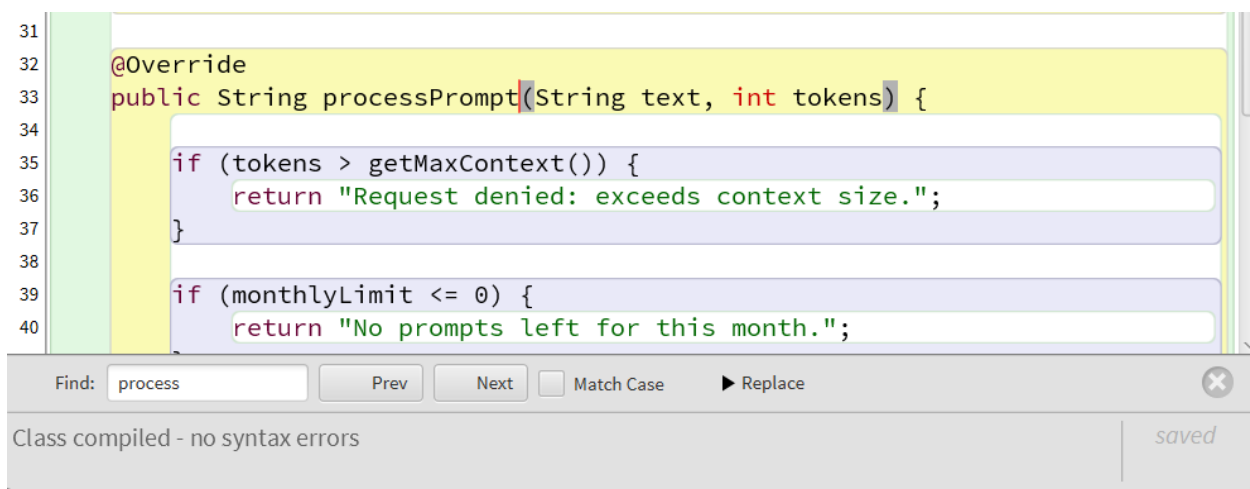


Figure 31 : Screenshot of solution to syntax error

6.2 Logical Error

Screenshot of Error:

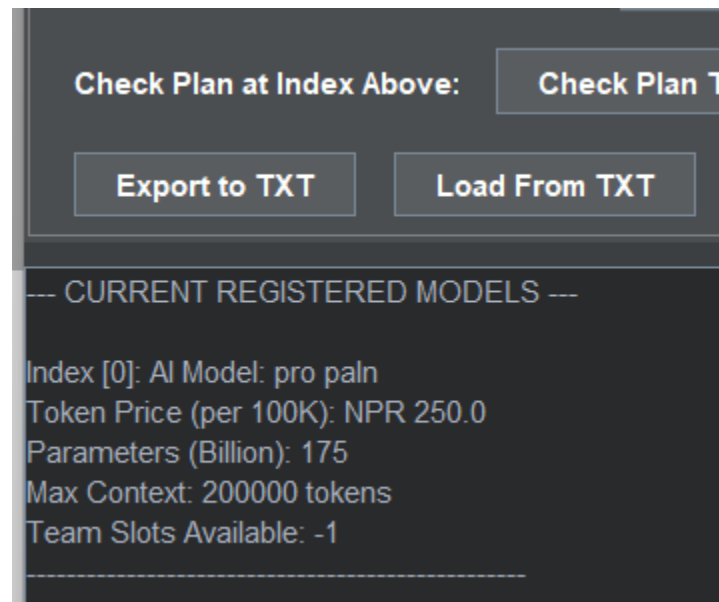


Figure 32 : Logical error encounter

Cause of Error:

In the ProPlan include member method, the program keeps allowing adding additional members even after the team capacity reached zero. Results in negative slots.

Solution:

Added an if statement block to check if team capacity is greater than 0 before adding a member. If the capacity is zero the program now says Team is full.

```
// add team member
public String includeMember(String member) {
    if (member == null || member.trim().isEmpty()) {
        return "Invalid member name.";
    }

    if (teamCapacity <= 0) {
        return "Team is full. Cannot add more members.";
    }

    teamCapacity--;
    return member + " successfully joined the team.";
}
```

6.3 Runtime Error

Screenshot of Error:

```
C:\Users\asus\Desktop\pw-coursework milestone 2\Apple-copy>java SubscriptionGUI
Exception in thread "AWT-EventQueue-0" java.lang.NumberFormatException: For input string: ""
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
```

Figure 33 : Runtime error

Cause:

The error occurred because the code attempt to convert a string into numeric type using an empty space into a number, the program throws this exception and crashes.

Solution:

I wrapped the code in a try-catch block. By encountering the exception, the program no longer crashes.

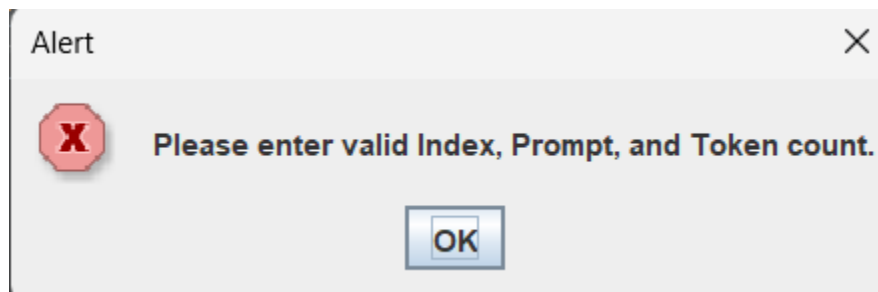


Figure 34 :Exception handling

7 Conclusion

Surely enough, the implementation of the AI Model Subscription Management System will become successful due to the use of Object-Oriented Programming together with Graphical User Interface. First of all, it should be said that in terms of the concepts of Object-Oriented Programming, which were applied throughout the process of developing the application, there is an enormous difference in approaches towards creating scripts and software solutions. Specifically, I had to rely on the principle of inheritance to build a proper hierarchy of classes, including AIModel. Moreover, using Polymorphism, I managed to create PersonalPlan and ProPlan classes, which allowed me to process subscription dynamically.

What concerns the problems that I faced while working on the project, such as those concerning software construction, Robustness and Data Integrity might be considered. Specifically, it became clear to me that Syntax, Runtime and Logical errors could pose a threat to the performance of the software created. Therefore, implementing try-catch statements together with checking the range of NumberFormatException and ArrayList indices made it possible for me to make the solution more robust and perform its professional operations. Besides, File I/O can be accessed later on by other parties

8 Bibliography

codecademy. (2024, 02 14). Retrieved from codecademy:

<https://www.codecademy.com/resources/blog/pseudocode>

imgguru. (2026, 05 11). Retrieved from imgguru: <https://www.imgguru.com/blog/what-is-java>

mivocloude. (2023, 12 06). Retrieved from mivocloude:

<https://mivocloud.com/blog/Bluej-or-the-IDE-that-your-developers-need>

Werner, C. (2019, 12 5). *ScienceDirect*. Retrieved from ScienceDirect:

<https://www.sciencedirect.com/topics/computer-science/method-definition>

9 Appendix

9.1 AIModel.java

```
/**
 * Abstract class for all model plan
 * @author (Shrawan karki)
 */
import java.io.Serializable;

public abstract class AIModel implements Serializable {

    private String name;
    private double tokenPrice;
    private int parameters;
    private int maxContext;

    public AIModel(String name, double tokenPrice,
                    int parameters, int maxContext) {

        this.name = name;
        this.tokenPrice = tokenPrice;
        this.parameters = parameters;
        this.maxContext = maxContext;
    }

    //getter methods
    public String getName() {
```

```
        return name;
    }

    public double getTokenPrice() {
        return tokenPrice;
    }

    public int getParameters() {
        return parameters;
    }

    public int getMaxContext() {
        return maxContext;
    }

    // token estimation logic
    private int estimateTokens(String text) {
        return text.length() / 4;
    }

    // token usage calculation
    public boolean calculateTokenUsage(String text, int outputTokens) {

        int inputTokens = estimateTokens(text);
        int totalTokens = inputTokens + 50 + outputTokens;

        return totalTokens <= maxContext;
    }
```

```

// report of token
public String getTokenReport(String text, int outputTokens) {

    int inputTokens = estimateTokens(text);
    int totalTokens = inputTokens + 50 + outputTokens;

    String report = "==== Token Usage Report =====\n";
    report += "Input Tokens: " + inputTokens + "\n";
    report += "Output Tokens: " + outputTokens + "\n";
    report += "System Overhead: 50\n";
    report += "Total Tokens: " + totalTokens + "\n";
    report += "Max Context: " + maxContext + "\n";

    if (totalTokens <= maxContext) {
        report += "Status: WITHIN LIMIT ";
    } else {
        report += "Status: EXCEEDS LIMIT ";
    }

    return report;
}

// abstract method
public abstract String processPrompt(String text, int tokens);

// display details
public String showDetails() {

```

```

    return "AI Model: " + name +
        "\nToken Price (per 100K): NPR " + tokenPrice +
        "\nParameters (Billion): " + parameters +
        "\nMax Context: " + maxContext + " tokens";
}
}

```

9.2 PersonalPlan.java

```

/**
 * Personal plan model child class of AIModel
 */
public class PersonalPlan extends AIModel {

    private int monthlyLimit;

    public PersonalPlan(String name, double tokenPrice,
        int parameters, int maxContext,
        int monthlyLimit) {

        super(name, tokenPrice, parameters, maxContext);
        this.monthlyLimit = monthlyLimit;
    }

    // checking remaining quota
    public int remainingQuota() {
        return monthlyLimit;
    }
}

```

```

// adding extra quota
public String addMorePrompts(int extra) {

    if (extra <= 0) {
        return "Invalid input. Prompts must be greater than zero.";
    }

    monthlyLimit += extra;
    return "Quota updated. Current prompts: " + monthlyLimit;
}

@Override
public String processPrompt(String text, int tokens) {

    if (tokens > getMaxContext()) {
        return "Request denied: exceeds context size.";
    }

    if (monthlyLimit <= 0) {
        return "No prompts left for this month.";
    }

    monthlyLimit--;

    return "Processing Prompt...\n" +
        "Text: " + text + "\n" +
        "Tokens Used: " + tokens + "\n" +

```

```

        "Remaining Quota: " + monthlyLimit;
    }

    @Override
    public String showDetails() {
        return super.showDetails() +
            "\nMonthly Prompts Remaining: " + monthlyLimit;
    }
}

```

9.3 ProPlan.java

```

/**
 * pro plan model Child class of AIModel
 */
public class ProPlan extends AIModel {

    private int teamCapacity;

    public ProPlan(String name, double tokenPrice,
        int parameters, int maxContext,
        int teamCapacity) {

        super(name, tokenPrice, parameters, maxContext);
        this.teamCapacity = teamCapacity;
    }

    // checking available slots
    public int slotsLeft() {

```

```
        return teamCapacity;
    }

    // add team member
    public String includeMember(String member) {

        if (member == null || member.trim().isEmpty()) {
            return "Invalid member name.";
        }

        if (teamCapacity <= 0) {
            return "Team is full. Cannot add more members.";
        }

        teamCapacity--;
        return member + " successfully joined the team.";
    }

    // remove member
    public String excludeMember(String member) {

        if (member == null || member.trim().isEmpty()) {
            return "Invalid member name.";
        }

        teamCapacity++;
        return member + " removed from team.";
    }
}
```

```
@Override
```

```
public String processPrompt(String text, int tokens) {

    if (tokens > getMaxContext()) {
        return "Token request exceeds model capacity.";
    }

    return "Pro Model Response Generated\n" +
        "Input: " + text + "\n" +
        "Token Count: " + tokens + "\n" +
        "(No monthly restriction)";
}
```

```
@Override
```

```
public String showDetails() {
    return super.showDetails() +
        "\nTeam Slots Available: " + teamCapacity;
}
}
```

9.4 SubscriptionGUI.java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import java.io.*;
```

```

public class SubscriptionGUI extends JFrame {
    private ArrayList<AIModel> models = new ArrayList<>();
    private final String TXT_FILE = "subscriptions.txt";

    // Theme Colors
    private final Color BACKGROUND_GREY = new Color(60, 63, 65);
    private final Color PANEL_GREY = new Color(75, 78, 80);
    private final Color TEXT_WHITE = Color.WHITE;
    private final Color BUTTON_COLOR = new Color(90, 93, 95);

    // UI Fields
    private JTextField nameField, priceField, paramField, contextField;
    private JTextField quotaField, teamField, indexField, promptField, tokenField,
    memberField;
    private JTextArea outputArea;

    public SubscriptionGUI() {
        setTitle("AI Model Subscription System - Shrawan karki");
        setSize(1100, 950);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(15, 15));
        getContentPane().setBackground(BACKGROUND_GREY);

        // GLOBAL configurations
        JPanel globalPanel = new JPanel(new GridLayout(2, 4, 10, 10));
        setupPanel(globalPanel, " Model Details ");
        nameField = new JTextField(); priceField = new JTextField();
        paramField = new JTextField(); contextField = new JTextField();
    }

```

```
addLabelAndField(globalPanel, "Model Name:", nameField);
addLabelAndField(globalPanel, "Price (NPR):", priceField);
addLabelAndField(globalPanel, "Parameters (B):", paramField);
addLabelAndField(globalPanel, "Max Context:", contextField);
```

```
JPanel hubsPanel = new JPanel(new GridLayout(1, 2, 20, 0));
hubsPanel.setOpaque(false);
```

```
JPanel personalHub = new JPanel(new GridLayout(3, 1, 10, 10));
setupPanel(personalHub, " Personal Plan ");
quotaField = new JTextField();
JButton btnAddPersonal = new JButton("Add Personal Plan");
styleButton(btnAddPersonal);
personalHub.add(createWhiteLabel("Monthly Prompt Limit:"));
personalHub.add(quotaField);
personalHub.add(btnAddPersonal);
```

```
JPanel proHub = new JPanel(new GridLayout(3, 1, 10, 10));
setupPanel(proHub, " Pro Plan ");
teamField = new JTextField();
JButton btnAddPro = new JButton("Add Pro Plan");
styleButton(btnAddPro);
proHub.add(createWhiteLabel("Team Capacity:"));
proHub.add(teamField);
proHub.add(btnAddPro);
```

```
hubsPanel.add(personalHub); hubsPanel.add(proHub);
```

```
// --- OPERATIONS & FILE MANAGEMENT ---

JPanel bottomContainer = new JPanel(new BorderLayout(10, 10));
bottomContainer.setOpaque(false);

JPanel opsMasterPanel = new JPanel(new GridLayout(4, 1, 0, 5));
setupPanel(opsMasterPanel, " Operations and File Management ");

// Row 1: Prompting
JPanel row1 = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 5));
row1.setOpaque(false);
indexField = new JTextField(5); promptField = new JTextField(15);
tokenField = new JTextField(5);
JButton btnPrompt = new JButton("Process Prompt");
styleButton(btnPrompt);
row1.add(createWhiteLabel("Target Index:"));
row1.add(indexField);
row1.add(createWhiteLabel("Prompt:"));
row1.add(promptField);
row1.add(createWhiteLabel("Tokens:"));
row1.add(tokenField);
row1.add(btnPrompt);

// Row 2: Member Management
JPanel row2 = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 5));
row2.setOpaque(false);
memberField = new JTextField(10);
JButton btnAddMember = new JButton("Add Member");
```

```

styleButton(btnAddMember);
row2.add(createWhiteLabel("Member Name:"));
row2.add(memberField);
row2.add(btnAddMember);

// Row 3: Check Type
JPanel row3 = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 5));
row3.setOpaque(false);
JButton btnCheckType = new JButton("Check Plan Type");
styleButton(btnCheckType);
row3.add(createWhiteLabel("Check Plan at Index Above:"));
row3.add(btnCheckType);

// Row 4: File I/O
JPanel row4 = new JPanel(new FlowLayout(FlowLayout.LEFT, 15, 5));
row4.setOpaque(false);
JButton btnExport = new JButton("Export to TXT");
JButton btnLoad = new JButton("Load From TXT");
JButton btnDisplay = new JButton("Display All");
JButton btnClear = new JButton("Clear All Fields");
styleButton(btnExport); styleButton(btnLoad);
styleButton(btnDisplay); styleButton(btnClear);
row4.add(btnExport); row4.add(btnLoad);
row4.add(new JLabel(" | "));
row4.add(btnDisplay);
row4.add(btnClear);

opsMasterPanel.add(row1);

```

```

opsMasterPanel.add(row2);
opsMasterPanel.add(row3);
opsMasterPanel.add(row4);

outputArea = new JTextArea(12, 50);
outputArea.setEditable(false);
outputArea.setBackground(new Color(40, 42, 44));
outputArea.setForeground(new Color(171, 178, 191));
JScrollPane scroll = new JScrollPane(outputArea);

bottomContainer.add(opsMasterPanel, BorderLayout.NORTH);
bottomContainer.add(scroll, BorderLayout.CENTER);

JPanel topWrapper = new JPanel(new BorderLayout(10, 10));
topWrapper.setOpaque(false);
topWrapper.add(globalPanel, BorderLayout.NORTH);
topWrapper.add(hubsPanel, BorderLayout.CENTER);
add(topWrapper, BorderLayout.NORTH);
add(bottomContainer, BorderLayout.CENTER);

// --- ACTION LISTENERS ---

btnAddPersonal.addActionListener(e -> addPersonalLogic());
btnAddPro.addActionListener(e -> addProLogic());
btnPrompt.addActionListener(e -> processPromptLogic());
btnAddMember.addActionListener(e -> addMemberLogic());

btnCheckType.addActionListener(e -> {

```

```

try {
    int idx = Integer.parseInt(indexField.getText());
    checkPlanType(idx);
} catch (NumberFormatException ex) { showError("Please enter a valid numeric
Index first!"); }
});

```

```

btnExport.addActionListener(e -> {
    if (models.isEmpty()) { showError("Nothing to export!"); return; }
    try (PrintWriter writer = new PrintWriter(new FileWriter(TXT_FILE))) {
        for (AIModel m : models) {
            writer.println(m.showDetails());
            writer.println("-----");
        }
        JOptionPane.showMessageDialog(null, "Data Exported to TXT!");
    } catch (IOException ex) { showError("Export Failed!"); }
});

```

```

btnLoad.addActionListener(e -> {
    try (BufferedReader reader = new BufferedReader(new FileReader(TXT_FILE)))
    {
        outputArea.setText("--- RETRIEVED FROM FILE ---\n\n");
        String line;
        while ((line = reader.readLine()) != null) { outputArea.append(line + "\n"); }
    } catch (Exception ex) { showError("Load Error: File not found."); }
});

```

```

btnDisplay.addActionListener(e -> displayLogic());
btnClear.addActionListener(e -> clearLogic());

```

```

    setVisible(true);
}

```

```

// --- FULL LOGIC IMPLEMENTATION ---

```

```

private void addPersonalLogic() {
    try {
        String name = nameField.getText();
        double price = Double.parseDouble(priceField.getText());
        int params = Integer.parseInt(paramField.getText());
        int context = Integer.parseInt(contextField.getText());
        int quota = Integer.parseInt(quotaField.getText());

        if (name.isEmpty()) throw new Exception("Name cannot be empty");

        models.add(new PersonalPlan(name, price, params, context, quota));
        outputArea.setText("SYSTEM: Personal Plan added successfully at Index " +
            (models.size() - 1));
    } catch (Exception ex) {
        showError("Invalid Input! Please fill all Model details and Monthly Limit
correctly.");
    }
}

```

```

private void addProLogic() {
    try {
        String name = nameField.getText();
        double price = Double.parseDouble(priceField.getText());

```

```

int params = Integer.parseInt(paramField.getText());
int context = Integer.parseInt(contextField.getText());
int teamCap = Integer.parseInt(teamField.getText());

if (name.isEmpty()) throw new Exception("Name cannot be empty");

models.add(new ProPlan(name, price, params, context, teamCap));
outputArea.setText("SYSTEM: Pro Plan added successfully at Index " +
(models.size() - 1));
    } catch (Exception ex) {
        showError("Invalid Input! Please fill all Model details and Team Capacity
correctly.");
    }
}

private void processPromptLogic() {
    try {
        int idx = Integer.parseInt(indexField.getText());
        String prompt = promptField.getText();
        int tokens = Integer.parseInt(tokenField.getText());

        if (idx >= 0 && idx < models.size()) {
            String result = models.get(idx).processPrompt(prompt, tokens);
            outputArea.setText("LOG: Processing Index " + idx + "...\\n" + result);
        } else {
            showError("Index out of range!");
        }
    } catch (Exception ex) {
        showError("Please enter valid Index, Prompt, and Token count.");
    }
}

```

```

    }
}

```

```

private void addMemberLogic() {
    try {
        int idx = Integer.parseInt(indexField.getText());
        String memberName = memberField.getText();

        if (idx >= 0 && idx < models.size()) {
            AIModel selected = models.get(idx);
            if (selected instanceof ProPlan) {
                String result = ((ProPlan) selected).includeMember(memberName);
                outputArea.setText(result);
            } else {
                showError("This index is a Personal Plan. Members can only be added to
Pro Plans!");
            }
        } else {
            showError("Invalid Index!");
        }
    } catch (Exception ex) {
        showError("Enter a numeric Index and a Member Name.");
    }
}

```

```

private void displayLogic() {
    if (models.isEmpty()) {
        outputArea.setText("The list is currently empty.");
        return;
    }
}

```

```

    }
    outputArea.setText("--- CURRENT REGISTERED MODELS ---\n\n");
    for (int i = 0; i < models.size(); i++) {
        outputArea.append("Index [" + i + "]: " + models.get(i).showDetails() + "\n");
        outputArea.append("-----\n");
    }
}

```

```

private void clearLogic() {
    nameField.setText("");
    priceField.setText("");
    paramField.setText("");
    contextField.setText("");
    quotaField.setText("");
    teamField.setText("");
    indexField.setText("");
    promptField.setText("");
    tokenField.setText("");
    memberField.setText("");
    outputArea.setText("");
}

```

```

private void checkPlanType(int index) {
    if (index < 0 || index >= models.size()) {
        showError("Index Out of Bounds! No plan exists at index " + index);
        return;
    }
    AIModel selectedPlan = models.get(index);
}

```

```

    if (selectedPlan instanceof PersonalPlan) {
        outputArea.setText("RESULT: Index " + index + " is a [ Personal Plan ].");
        JOptionPane.showMessageDialog(this, "Plan Type: Personal Plan");
    } else if (selectedPlan instanceof ProPlan) {
        outputArea.setText("RESULT: Index " + index + " is a [ Pro Plan ].");
        JOptionPane.showMessageDialog(this, "Plan Type: Pro Plan");
    }
}

// Styling Helpers
private void setupPanel(JPanel p, String title) {
    p.setBackground(PANEL_GREY);

    p.setBorder(BorderFactory.createTitledBorder(BorderFactory.createLineBorder(Color.GRAY), title, 0, 0, null, TEXT_WHITE));
}

private void styleButton(JButton b) { b.setBackground(BUTTON_COLOR);
    b.setForeground(TEXT_WHITE); b.setFocusPainted(false);
}

private JLabel createWhiteLabel(String text) { JLabel l = new JLabel(text);
    l.setForeground(TEXT_WHITE); return l;
}

private void addLabelAndField(JPanel p, String text, JTextField f) {
    p.add(createWhiteLabel(text));
    p.add(f);
}

```

```
private void showError(String msg) { JOptionPane.showMessageDialog(this, msg,  
"Alert", JOptionPane.ERROR_MESSAGE); }
```

```
public static void main(String[] args) { new SubscriptionGUI(); }  
}
```